

Known Issues

1. STM32 firmware cannot be reliably updated from the SD card (use ST-LINK)

Summary

What works	ESP32 self-update from SD  · STM32 update via ST-LINK 
What doesn't	STM32 update from the SD card over UART  (unreliable)
Root cause	No free ESP32 GPIO is broken out to drive the STM32 <code>B00T0</code> pin, and the STM32G4 ROM bootloader is not USART-responsive after a software-only jump
Workaround	Flash the STM32 with an ST-LINK programmer
Future fix	Move the display tier to an ESP32-P4 (more usable GPIO) and add a <code>B00T0</code> control wire, or add a tiny GPIO expander / spare pin for <code>B00T0</code>

The feature, and what it's supposed to do

The product goal is a **fully self-contained field update**: drop a new `/stm32_firmware.bin` on the SD card, tap “Update STM32” on the touchscreen, and the ESP32 reflashes the STM32 over the existing UART link — no programmer, no PC. The ESP32 already does exactly this for its **own** firmware.

To flash the STM32 over UART, the chip must first be in its **factory ROM bootloader** (the ST AN3155 protocol). On a normal STM32 design you enter the ROM bootloader by **asserting the `B00T0` pin high and resetting** — the chip then starts the ROM bootloader instead of the application, and the bootloader listens on **USART1 (PA9/PA10) at 115200 8E1**.

That is where this hardware falls short.

Why it happens (the detailed root cause)

a) There is no `B00T0` control wire — and nowhere to add one

The STM32's `B00T0` is on **PB8** and is **not** wired to the ESP32. To drive it in software we'd need a spare ESP32 GPIO routed to PB8. The Sunton **ESP32-8048S043C** has essentially no free, broken-out GPIO:

GPIO(s)	Consumed by
1–9, 14–16, 21, 39–42, 45–48	RGB LCD parallel bus (16 data + DE/VSYNC/HSYNC/PCLK)
19, 20	GT911 capacitive touch I ² C (SDA / SCL)
17, 18	STM32 UART link (RX / TX)
10–13	microSD card SPI (CS / MOSI / SCK / MISO)
43, 44	USB-serial debug console (UART0)
0	BOOT button
26–37	SPI flash + PSRAM (octal)

Only pins **11/12/13, 17/18, 19/20, 43/44** reach a physical header (P1–P4), and **every one of them is already in use** — SD card, STM32 UART, touch, or debug console. The internally-free GPIOs (e.g. GPIO38) are **not bonded to any accessible pad**. So there is **no pin to wire to `B00T0`** without sacrificing the touchscreen, the SD card, or the debug console.

b) The software-only jump does not wake the G4 ROM bootloader

Because we can't touch `B00T0`, we tried entering the bootloader **purely in software**: the running app jumps to system memory (`0x1FFF0000`) via `VTOR` / `__set_MSP` / a memory remap, mimicking a `B00T0=1` reset.

On-hardware testing proved this **does not work on the STM32G4**:

- The STM32 received the `B00TLOADER` command, ACK'd it, and executed the jump (confirmed in the serial logs).
- But the ROM bootloader then **never responded** to the AN3155 `0x7F` auto-baud byte — no `0x79` ACK, on every one of 10 sync attempts.

This is a known STM32G4 behavior: the ROM bootloader expects a **true reset with `B00T0` asserted**, not a warm jump from a running application. After a software jump the USART interface frequently comes up dead on G4/L4/H7 (unlike F4, where the same trick usually works). The application's jump sequence is textbook — this is a **silicon / ROM limitation**, not a bug in our code.

Things that were verified along the way (so they're ruled out)

- **UART pins match** — the app and the ROM bootloader both use USART1 **PA9/PA10**; no remap mismatch.

- **Parity** — the ESP32 side correctly sends **8E1** (ESP-IDF `UART_PARITY_EVEN`); TX bytes confirmed leaving GPIO18.
- **Wiring** — the same two wires carry a working **2 Mbaud** app link in both directions, so 115200 is trivially within their capability.
- **Command parsing** — the STM32 parser matches `BOOTLOADER` exactly (`strcmp == 0`) and ACKs.
- **Watchdog** — the IWDG was disabled so it can't reset the chip out of the bootloader.

Every layer except “a software jump produces a responsive ROM bootloader” checks out. That one layer is the wall.

Workaround: flash the STM32 with an ST-LINK

This is the **supported, reliable** path and works every time.

1. Connect an **ST-LINK V2/V3** to the STM32 SWD header (SWDIO, SWCLK, GND, 3V3 reference).
2. Build the firmware:

```
bash
cd STM32G474CE
pio run -e g474ceu6_stlink
```

3. Flash it:

```
bash
pio run -e g474ceu6_stlink -t upload
...or load STM32G474CE/.pio/build/g474ceu6_stlink/firmware.bin at address
0x08000000 with STM32CubeProgrammer.
```

The **ESP32** continues to self-update from the SD card normally — only the STM32 needs the programmer.

Experimental (unverified) option-byte SD-flash path

The firmware also contains an **experimental** wire-free attempt: on the `BOOTLOADER` command the STM32 reprograms its **boot option bytes** (`nSWBOOT0=0`, `nBOOT0=1`, `nBOOT1=1`) and launches them, which performs a **real reset** into the ROM bootloader (electrically like `BOOT0=1`). The freshly flashed app then restores the option bytes (`nSWBOOT0=1`) on its first boot.

- **Status: UNVERIFIED on hardware.** It compiles and is logically sound, but has not been confirmed to complete a full flash cycle.
 - **Risk:** if the post-flash “Go”/launch ever fails, the option bytes stay in “force bootloader” mode and the chip will re-enter the bootloader on every reset until reflashed.
 - **Always keep an ST-LINK handy** — it is the guaranteed recovery path and will reset the option bytes back to normal.
-

Future solution

1. **ESP32-P4 display board (preferred).** The ESP32-P4 has substantially more usable GPIO. Moving Tier 2 to a P4-based panel frees pins to run a dedicated **B00T0 (and ideally NRST) control wire** to the STM32. That enables the **ST-supported hardware bootloader entry** (assert `B00T0` , pulse reset), which is rock-solid — and makes true wire-free SD updates of the STM32 possible.
2. **Spare pin / GPIO expander on a board respin.** If staying on an ESP32-S3 class panel, route one spare GPIO (or an I²C GPIO expander output) to STM32 `B00T0` during the next PCB revision.
3. **Validate the option-byte path.** With careful on-hardware testing and a guaranteed ST-LINK fallback, the experimental option-byte method could be promoted to the supported SD-flash path without any hardware change.

Until one of the above lands, **use an ST-LINK to update the STM32.**